Supersetid :8004607

Digital Nurture 5.0 Deep Skilling Handbook Java FSE - Solution

PL SQL Programming

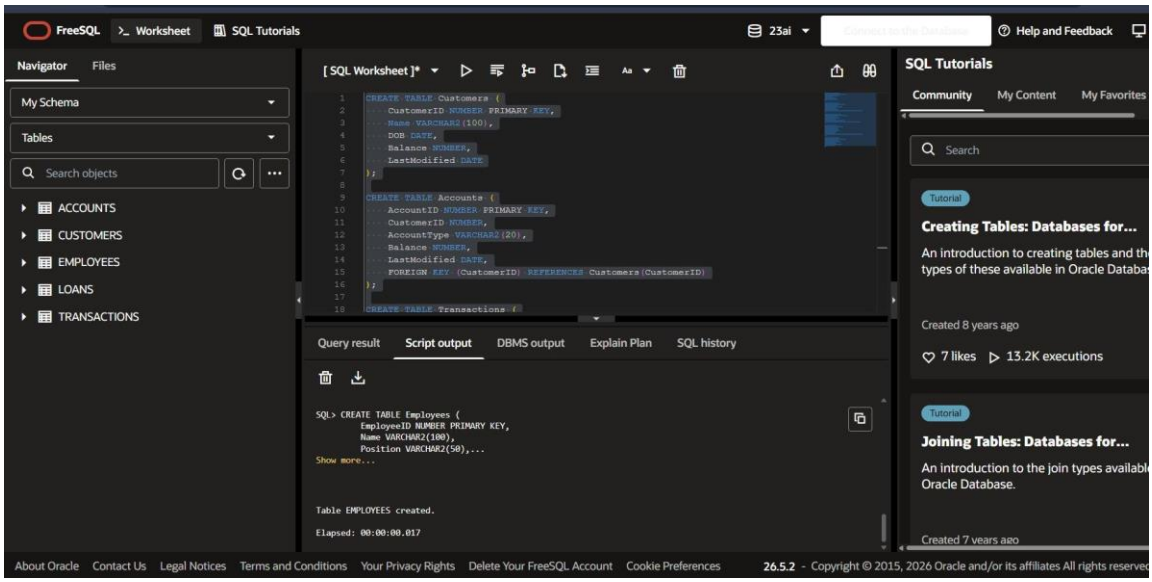
Supersetid :8004607

Table creation:

```
CREATE TABLE Customers (
   CustomerID NUMBER PRIMARY KEY,
   Name VARCHAR2(100),
   DOB DATE,
   Balance NUMBER,
   LastModified DATE
);

CREATE TABLE Accounts (
   AccountID NUMBER PRIMARY KEY,
   CustomerID NUMBER,
   AccountType VARCHAR2(20),
   Balance NUMBER,
   LastModified DATE,
   FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Transactions (
   TransactionID NUMBER PRIMARY KEY,
   AccountID NUMBER,
   TransactionDate DATE,
   Amount NUMBER,
   TransactionType VARCHAR2(10),
   FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

CREATE TABLE Loans (
   LoanID NUMBER PRIMARY KEY,
   CustomerID NUMBER,
   LoanAmount NUMBER,
   InterestRate NUMBER,
   StartDate DATE,
   EndDate DATE,
   FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE Employees (
   EmployeeID NUMBER PRIMARY KEY,
   Name VARCHAR2(100),
   Position VARCHAR2(50),
   Salary NUMBER,
   Department VARCHAR2(50),
   HireDate DATE
```

Supersetid :8004607

);



Value Insertion:

INSERT INTO Customers
VALUES (1,'John Doe',TO_DATE('1985-05-15','YYYY-MM-DD'),1000,SYSDATE);

INSERT INTO Customers
VALUES (2,'Jane Smith',TO_DATE('1990-07-20','YYYY-MM-DD'),1500,SYSDATE);

INSERT INTO Accounts
VALUES (1,1,'Savings',1000,SYSDATE);

INSERT INTO Accounts
VALUES (2,2,'Checking',1500,SYSDATE);

INSERT INTO Transactions
VALUES (1,1,SYSDATE,200,'Deposit');

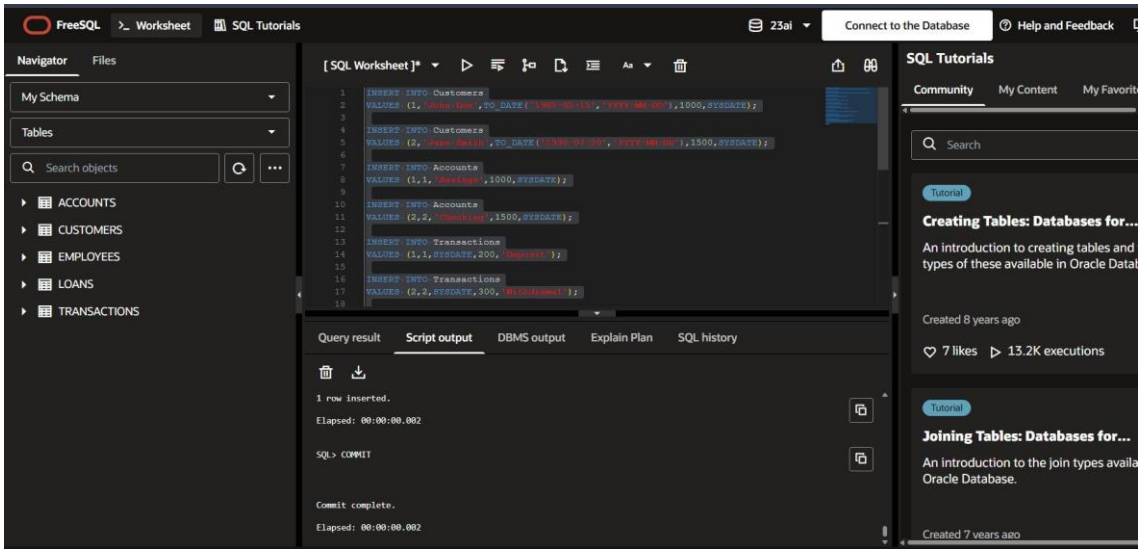
INSERT INTO Transactions
VALUES (2,2,SYSDATE,300,'Withdrawal');

INSERT INTO Loans
VALUES (1,1,5000,5,SYSDATE,ADD_MONTHS(SYSDATE,60));

INSERT INTO Employees
VALUES (1,'Alice Johnson','Manager',70000,'HR',
TO_DATE('2015-06-15','YYYY-MM-DD'));

INSERT INTO Employees

Supersetid :8004607



VALUES (2,'Bob Brown','Developer',60000,'IT',
TO_DATE('2017-03-20','YYYY-MM-DD'));

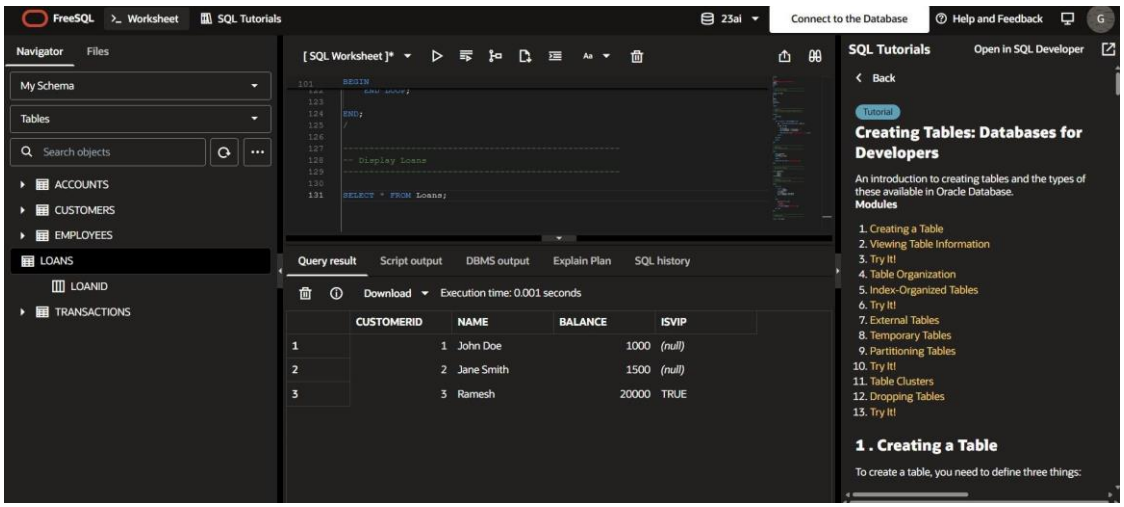
Exercise 1: Control Structures

```
SET SERVEROUTPUT ON;
INSERT INTO Customers
(CustomerID, Name, DOB, Balance, LastModified, IsVIP)VALUES(3,'Ramesh',TO_DATE('1950-01-10','YYYY-
MM-DD'),20000,SYSDATE,NULL);
INSERT INTO LoansVALUES(2,3,10000,8,SYSDATE,SYSDATE+20);
COMMIT;
-- Scenario 1
DECLARE
   age NUMBER;
BEGIN
   FOR c IN (SELECT * FROM Customers) LOOP
      age := TRUNC(MONTHS_BETWEEN(SYSDATE,c.DOB)/12);
      IF age > 60 THEN
         UPDATE Loans
         SET InterestRate = InterestRate - 1
         WHERE CustomerID = c.CustomerID;
         DBMS_OUTPUT.PUT_LINE('Discount Applied to '||c.Name);
      END IF;
   END LOOP;
   COMMIT;
END;
/
-- Scenario 2
BEGIN
   UPDATE Customers
   SET IsVIP='TRUE'
   WHERE Balance>10000;
   COMMIT;
   DBMS_OUTPUT.PUT_LINE('VIP Status Updated');
END;/
```

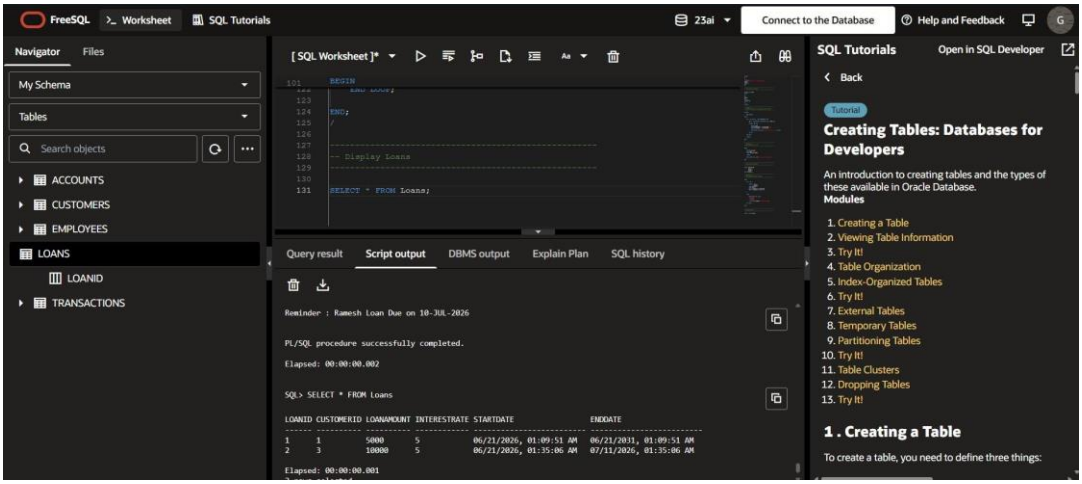
Supersetid :8004607

```
-- Display VIP Customers
SELECT CustomerID,
      Name,
      Balance,
      IsVIP
FROM Customers;
-- Scenario 3
BEGIN
   FOR r IN (
      SELECT c.Name,
            l.EndDate
      FROM Customers c
      JOIN Loans l
      ON c.CustomerID=l.CustomerID
      WHERE l.EndDate<=SYSDATE+30)
   LOOP
      DBMS_OUTPUT.PUT_LINE(
      'Reminder : '
      ||r.Name||
      ' Loan Due on '
      ||TO_CHAR(r.EndDate,'DD-MON-YYYY'));
   END LOOP;
END;
/
SELECT * FROM Loans;
```

OUTPUT:



Supersetid :8004607



Exercise 2: Error Handling

```
SET SERVEROUTPUT ON;
-- Scenario 1
-- Safe Transfer Funds
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
   fromAcc NUMBER,
   toAcc NUMBER,
   amount NUMBER
)
IS
   balance NUMBER;
BEGIN
   SELECT Balance
   INTO balance
   FROM Accounts
   WHERE AccountID = fromAcc;
   IF balance < amount THEN
      DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
      ROLLBACK;
   ELSE
      UPDATE Accounts
      SET Balance = Balance - amount
      WHERE AccountID = fromAcc;
      UPDATE Accounts
      SET Balance = Balance + amount
      WHERE AccountID = toAcc;
      COMMIT;
      DBMS_OUTPUT.PUT_LINE('Amount Transferred Successfully');
   END IF;

EXCEPTION
   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error : ' || SQLERRM);
```

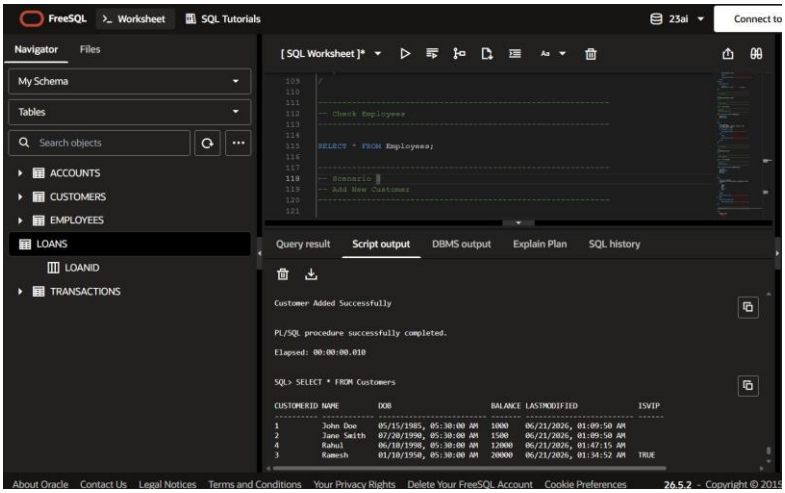
Supersetid :8004607

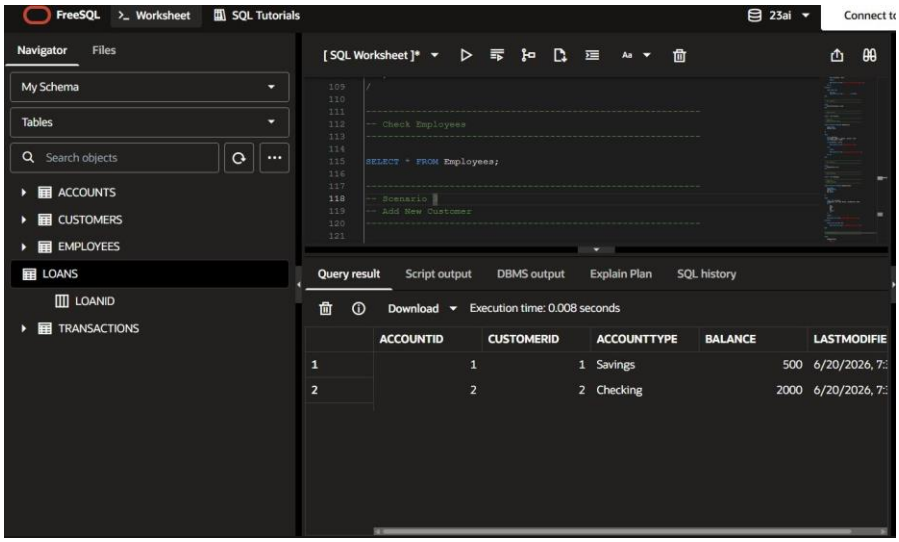
```
END;
/
-- Test Scenario 1
BEGIN
   SafeTransferFunds(1,2,500);
END;
/
SELECT * FROM Accounts;
-- Scenario 2
-- Update Employee Salary
CREATE OR REPLACE PROCEDURE UpdateSalary(
   empId NUMBER,
   percent NUMBER
)
IS
BEGIN
   UPDATE Employees
   SET Salary = Salary + (Salary * percent / 100)
   WHERE EmployeeID = empId;
   IF SQL%ROWCOUNT = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Employee Not Found');
   ELSE
      COMMIT;
      DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');
   END IF;
END;
/
--------------------------------------------------------------------
-- Test Scenario 2
--------------------------------------------------------------------

BEGIN
   UpdateSalary(1,10);
END;
/
SELECT * FROM Employees;
-- Scenario 3
-- Add New Customer
CREATE OR REPLACE PROCEDURE AddNewCustomer(
   cid NUMBER,
   cname VARCHAR2,
   dob DATE,
   bal NUMBER
)
IS
BEGIN

   INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified, IsVIP)VALUES(cid, cname, dob,
bal, SYSDATE,  NULL);
   COMMIT;
```

Supersetid :8004607

```
   DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');
EXCEPTION
   WHEN DUP_VAL_ON_INDEX THEN
      DBMS_OUTPUT.PUT_LINE('Customer ID Already Exists');
END;
/
-- Test Scenario 3
BEGIN
   AddNewCustomer(
      4,
      'Rahul',
      TO_DATE('1998-06-10','YYYY-MM-DD'),
      12000
   );
END;
/
-- Check Customers
SELECT * FROM Customers;
```

OUTPUT:





Supersetid :8004607

Exercise 3: Stored Procedures

```
SET SERVEROUTPUT ON;
--------------------------------------------------------------------
-- Scenario 1
-- Process Monthly Interest (1% for Savings Accounts)
--------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
   UPDATE Accounts
   SET Balance = Balance + (Balance * 0.01)
   WHERE AccountType = 'Savings';
   COMMIT;
   DBMS_OUTPUT.PUT_LINE('Monthly Interest Applied Successfully');
END;
/
-- Test Scenario 1
BEGIN
   ProcessMonthlyInterest;
END;
/
SELECT * FROM Accounts;
-- Scenario 2
-- Update Employee Bonus

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
   dept VARCHAR2,
   bonus NUMBER
)
IS
BEGIN
   UPDATE Employees
```

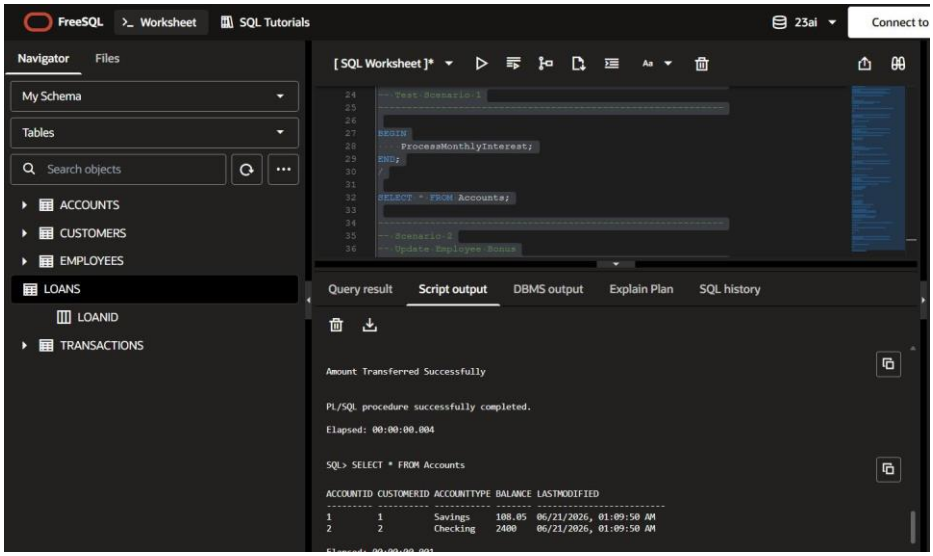
Supersetid :8004607

```
   SET Salary = Salary + (Salary * bonus / 100)
   WHERE Department = dept;
   COMMIT;
   DBMS_OUTPUT.PUT_LINE('Bonus Updated Successfully');
END;
/
-- Test Scenario 2
BEGIN
   UpdateEmployeeBonus('IT',10);
END;
/
SELECT * FROM Employees;
-- Scenario 3
-- Transfer Fund
CREATE OR REPLACE PROCEDURE TransferFunds(fromAccount NUMBER,toAccount NUMBER,amount
NUMBER)
IS
   balance NUMBER;
BEGIN
   SELECT Balance
   INTO balance
   FROM Accounts
   WHERE AccountID = fromAccount;
   IF balance >= amount THEN
      UPDATE Accounts
      SET Balance = Balance - amount
      WHERE AccountID = fromAccount;
      UPDATE AccountsSET Balance = Balance + amountWHERE AccountID = toAccount;
      COMMIT;  DBMS_OUTPUT.PUT_LINE('Amount Transferred Successfully');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
   END IF;
END;
/
```

Supersetid :8004607

```
-- Test Scenario 3
BEGIN
   TransferFunds(1,2,200);
END;
/
SELECT * FROM Accounts;
```

OUTPUT:



Exercise 4: Functions

```
SET SERVEROUTPUT ON;
-- Scenario 1


CREATE OR REPLACE FUNCTION CalculateAge(
   dob DATE
)
RETURN NUMBER
IS
   age NUMBER;
```

Supersetid :8004607

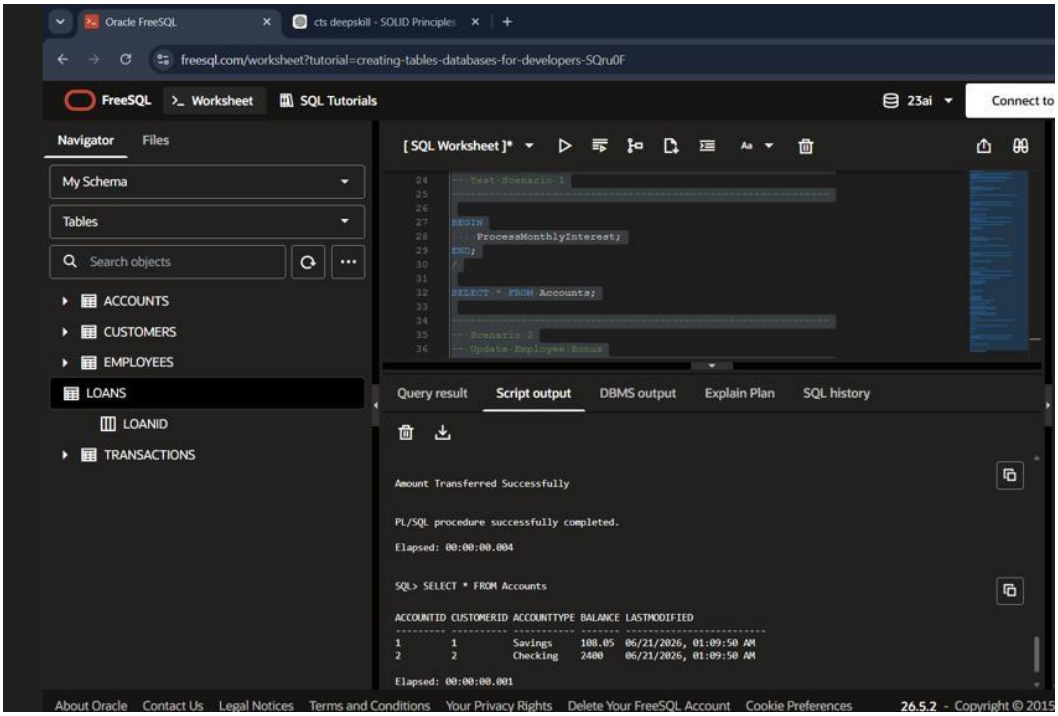
```
BEGIN

   age := TRUNC(MONTHS_BETWEEN(SYSDATE, dob) / 12);

   RETURN age;

END;
/
-----------------------------------------------------------------------
-- Test Scenario 1
-----------------------------------------------------------------------
SELECT Name,
      CalculateAge(DOB) AS Age
FROM Customers;
-----------------------------------------------------------------------
-- Scenario 2
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(m
   loanAmount NUMBER,
   interestRate NUMBER,
   years NUMBER

)
RETURN NUMBER
IS
   monthlyInstallment NUMBER;
BEGIN
   monthlyInstallment :=
   (loanAmount + (loanAmount * interestRate * years /100))
   /(years*12);
   RETURN monthl
-- Test Scenario 2
-----------------------------------------------------------------------
SELECT LoanID,
      CalculateMonthlyInstallment(LoanAmount,InterestRate,5)
```

Supersetid :8004607

```
    AS Monthly_Installment
FROM Loans;
-- Scenario 3
CREATE OR REPLACE FUNCTION HasSufficientBalance(
   accId NUMBER,
   amount NUMBER
)
RETURN BOOLEAN
IS
   balance NUMBER;
BEGIN
   SELECT Balance
   INTO balance
   FROM Accounts
   WHERE AccountID = accId;
   IF balance >= amount THEN
      RETURN TRUE;
   ELSE
      RETURN FALSE;
   END IF;
END;
/
-- Test Scenario 3
DECLARE
   result BOOLEAN;
BEGIN
   result := HasSufficientBalance(1,500);
   IF result THEN
      DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
   END IF;

END;
```

Supersetid :8004607

OUTPUT:



Exercises 5 Trigger

```
SET SERVEROUTPUT ON;

--------------------------------------------------------------------

-- Scenario 1

CREATE OR REPLACE TRIGGER UpdateCustomerLastModified

BEFORE UPDATE

ON Customers

FOR EACH ROW

BEGIN

   :NEW.LastModified := SYSDATE;

END;

/

-- Test Scenario 1

UPDATE Customers

SET Balance = 5000

WHERE CustomerID = 1;

COMMIT;
```

Supersetid :8004607

```
SELECT * FROM Customers;
-- Scenario 2
-- Create AuditLog Table
CREATE TABLE AuditLog(
   LogID NUMBER GENERATED BY DEFAULT AS IDENTITY,
   TransactionID NUMBER,
   AccountID NUMBER,
   Amount NUMBER,
   ActionDate DATE
);
-- Trigger to Log Transactions
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT
ON Transactions
FOR EACH ROW
BEGIN
   INSERT INTO AuditLog
   (TransactionID,AccountID,Amount,ActionDate)
   VALUES
   (:NEW.TransactionID,
    :NEW.AccountID,
    :NEW.Amount,
    SYSDATE);
END;
/
-- Test Scenario 2
INSERT INTO Transactions
VALUES
(
3,
1,
SYSDATE,
```

Supersetid :8004607

```
500,
'Deposit'
);
COMMIT;
SELECT * FROM AuditLog;
-- Scenario 3
-- Check Deposit and Withdrawal Rules
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT
ON Transactions
FOR EACH ROW
DECLARE
   balance NUMBER;
BEGIN
   SELECT Balance
   INTO balance
   FROM Accounts
   WHERE AccountID = :NEW.AccountID;
   IF :NEW.TransactionType='Withdrawal'
     AND :NEW.Amount > balance THEN
       RAISE_APPLICATION_ERROR(
       -20001,
       'Insufficient Balance');
   END IF;
   IF :NEW.TransactionType='Deposit'
     AND :NEW.Amount<=0 THEN

       RAISE_APPLICATION_ERROR(
       -20002,
       'Deposit Amount Must Be Positive');
   END IF;
END;
```

Supersetid :8004607

/

INSERT INTO Transactions

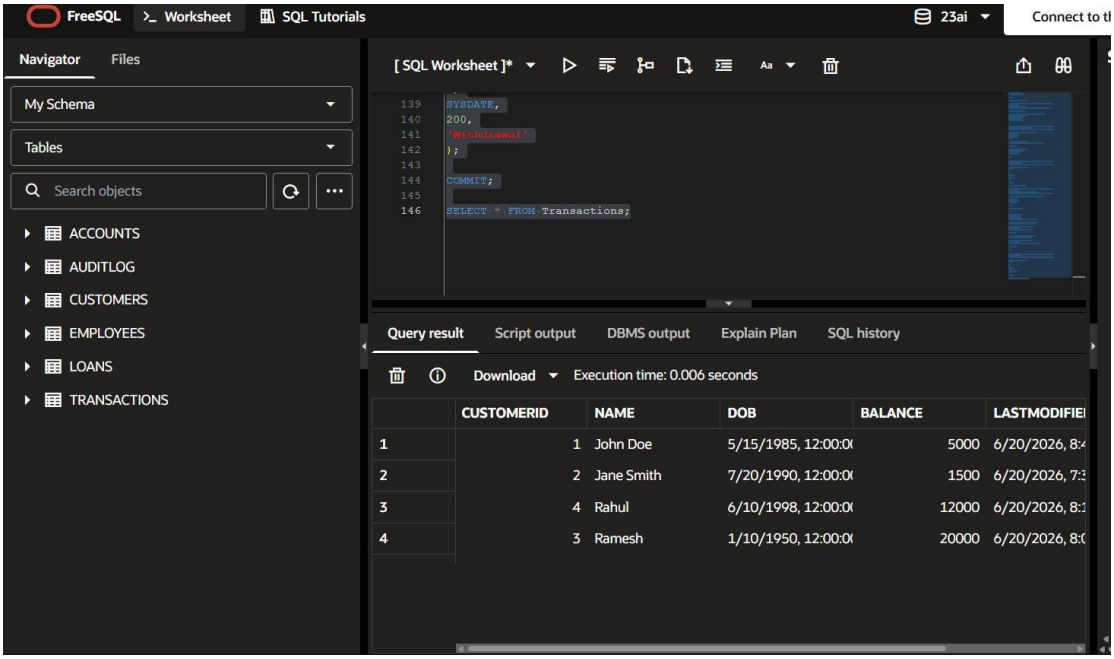
VALUES

(4,1,SYSDATE,200,'Withdrawal');

COMMIT;

SELECT * FROM Trans;

OUTPUT:



Exercises 6 cursorss

SET SERVEROUTPUT ON;

-- Scenario 1: Generate Monthly Statements

DECLARE

CURSOR GenerateMonthlyStatements IS

SELECT TransactionID,

AccountID,

Amount,

TransactionType

FROM Transactions

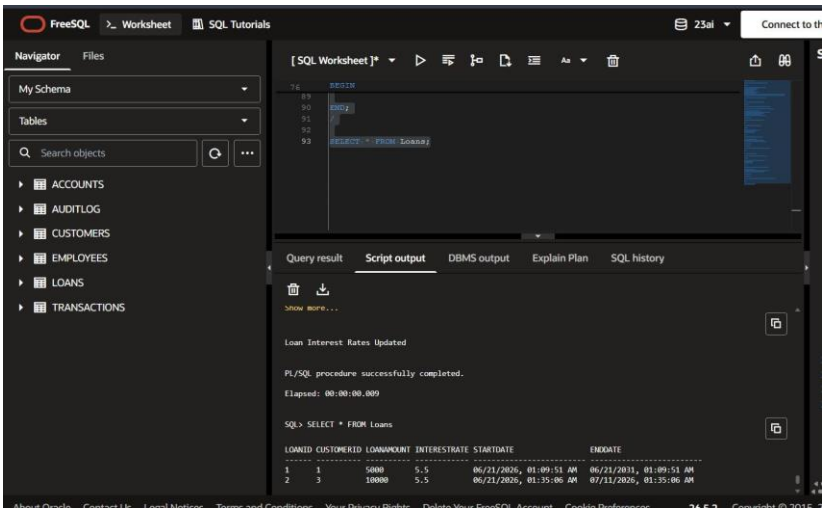
Supersetid :8004607

```
   WHERE EXTRACT(MONTH FROM TransactionDate)=EXTRACT(MONTH FROM SYSDATE)
     AND EXTRACT(YEAR FROM TransactionDate)=EXTRACT(YEAR FROM SYSDATE);
   t GenerateMonthlyStatements%ROWTYPE;
BEGIN
   OPEN GenerateMonthlyStatements;
   LOOP
      FETCH GenerateMonthlyStatements INTO t;

      EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
      'Transaction ID : '||t.TransactionID||
      ' Account : '||t.AccountID||
      ' Amount : '||t.Amount||
      ' Type : '||t.TransactionType);
   END LOOP;
   CLOSE GenerateMonthlyStatements;
END;
/
-- Scenario 2: Apply Annual Fee
DECLARE
   CURSOR ApplyAnnualFee IS
   SELECT AccountID
   FROM Accounts;
BEGIN
   FOR acc IN ApplyAnnualFee LOOP
      UPDATE Accounts
      SET Balance = Balance - 100
      WHERE AccountID = acc.AccountID;
   END LOOP;
```

Supersetid :8004607

```
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Annual Fee Applied');
END;
/SELECT * FROM Accounts;
-- Scenario 3: Update Loan Interest Rates
DECLARE
    CURSOR UpdateLoanInterestRates IS
    SELECT LoanID FROM Loans;
BEGIN
    FOR loan IN UpdateLoanInterestRates LOOP
        UPDATE Loans
        SET InterestRate = InterestRate + 0.5
        WHERE LoanID = loan.LoanID;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Loan Interest Rates Updated');
END;
/
SELECT * FROM Loans;
```

OUTPUT:



Supersetid :8004607

Exercises 7 Packages

```
SET SERVEROUTPUT ON;
-- Scenario 1: Customer Management Package
CREATE OR REPLACE PACKAGE CustomerManagement AS
   PROCEDURE AddCustomer(
      cid NUMBER,
      cname VARCHAR2,
      dob DATE,
      bal NUMBER
   );
   PROCEDURE UpdateCustomer(
      cid NUMBER,
      newBalance NUMBER
   );
   FUNCTION GetBalance(
      cid NUMBER
   ) RETURN NUMBER;
END CustomerManagement;
/
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
   PROCEDURE AddCustomer(
      cid NUMBER,
      cname VARCHAR2,
      dob DATE,
      bal NUMBER
   )
   IS   BEGIN
      INSERT INTO Customers
      (CustomerID,Name,DOB,Balance,LastModified,IsVIP)
      VALUES
      (cid,cname,dob,bal,SYSDATE,NULL);
      COMMIT;
```

Supersetid :8004607

```
  END;
  PROCEDURE UpdateCustomer(
    cid NUMBER, newBalance NUMBER  )
  IS  BEGIN
    UPDATE Customers
    SET Balance = newBalance
    WHERE CustomerID = cid;
 COMMIT;
  END;
  FUNCTION GetBalance(  cid NUMBER   )
  RETURN NUMBER  IS
    bal NUMBER;
  BEGIN
    SELECT Balance
    INTO bal
    FROM Customers
    WHERE CustomerID = cid;
    RETURN bal;
  END;
END CustomerManagement;
/
BEGIN
  CustomerManagement.AddCustomer(5,'Arun', TO_DATE('1999-05-10','YYYY-MM-DD'), 5000 );
 CustomerManagement.UpdateCustomer(5,8000);
END;
/
SELECT CustomerManagement.GetBalance(5) AS Balance
FROM Dual;
-- Scenario 2: Employee Management Package
CREATE OR REPLACE PACKAGE EmployeeManagement AS
  PROCEDURE HireEmployee(
    id NUMBER, name VARCHAR2, pos VARCHAR2, sal NUMBER, dept VARCHAR2
```

Supersetid :8004607

```
   );  PROCEDURE UpdateEmployee(
      id NUMBER,   newsalary NUMBER
   );
   FUNCTION AnnualSalary(
      id NUMBER
   ) RETURN NUMBER;
END EmployeeManagement;
/
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
   PROCEDURE HireEmployee(
      id NUMBER,
      name VARCHAR2,
      pos VARCHAR2,
      sal NUMBER,
      dept VARCHAR2
   )
   IS  BEGIN
      INSERT INTO Employees
      VALUES(id,name, pos, sal,    dept, SYSDATE);
COMMIT;
   END;
   PROCEDURE UpdateEmployee(
      id NUMBER,
      newsalary NUMBER  )
   IS  BEGIN
      UPDATE Employees
      SET Salary = newsalary
      WHERE EmployeeID=id;
      COMMIT;
   END;
FUNCTION AnnualSalary(  id NUMBER )
   RETURN NUMBER
```

Supersetid :8004607

```
   IS
      sal NUMBER;
   BEGIN
      SELECT Salary
      INTO sal
      FROM Employees
      WHERE EmployeeID=id;
      RETURN sal*12;
   END;
END EmployeeManagement;
/
BEGIN
   EmployeeManagement.HireEmployee(
   3,'David','Tester', 40000, 'IT'); EmployeeManagement.UpdateEmployee(3,45000);
END;
/
SELECT EmployeeManagement.AnnualSalary(3)
AS AnnualSalary
FROM Dual;
-- Scenario 3: Account Operations Package
CREATE OR REPLACE PACKAGE AccountOperations AS
   PROCEDURE OpenAccount(
      aid NUMBER, cid NUMBER,type VARCHAR2,bal NUMBER
   );
   PROCEDURE CloseAccount(
      aid NUMBER
   );
   FUNCTION TotalBalance( cid NUMBER )
    RETURN NUMBER;
END AccountOperations;
/
CREATE OR REPLACE PACKAGE BODY AccountOperations AS
```

Supersetid :8004607

```
  PROCEDURE OpenAccount(
     aid NUMBER,
     cid NUMBER,
     type VARCHAR2,
     bal NUMBER
  )
  IS  BEGIN
     INSERT INTO Accounts
     VALUES( aid,cid, type,bal, SYSDATE);
COMMIT;
  END;
  PROCEDURE CloseAccount(  aid NUMBER )
  IS
  BEGIN
     DELETE FROM Accounts
     WHERE  AccountID=aid;
     COMMIT;
  END;
  FUNCTION TotalBalance(  cid NUMBER  )
  RETURN NUMBER
  IS
     total NUMBER;
  BEGIN
     SELECT SUM(Balance)
     INTO total
     FROM Accounts
     WHERE CustomerID=cid;
     RETURN total;
  END;
END AccountOperations;
/
BEGIN
```

Supersetid :8004607

```
   AccountOperations.OpenAccount(
   5,
   1,
   'Savings',
   3000);


END;
/


SELECT AccountOperations.TotalBalance(1)
AS TotalBalance
FROM Dual;
BEGIN
   AccountOperations.CloseAccount(5);
END;
/
SELECT * FROM Accounts;
```

OUTPUT: